Out-of-Order Pipelines

Performance Equation

$$\frac{\#Programs}{\#Seconds} = \frac{\#Programs}{\#insts} * \frac{\#insts}{\#cycles} * \frac{\#cycles}{\#seconds}$$

$$= \frac{IPC * freq}{\#insts/program}$$

$$= \frac{IPC * freq}{\#insts}$$ (assume just 1 program)

- IPC is the number of instructions per cycle
- Let us loosely refer to the reciprocal of the time per program as the **performance**

So, what does performance depend on …

#instructions in the program

- Depends on the compiler

Frequency

- Depends on the transistor technology and the architecture
  - If we have more pipeline stages, then the time to traverse each stage reduces roughly proportionally
  - Given that each stage needs to be processed in one clock cycle, smaller the stage, higher the frequency
  - To increase the frequency, we simply need to increase the number of pipeline stages

IPC

- Depends on the architecture and the compiler

How to improve performance?

There are **3** factors:

- IPC, #instructions, and frequency
- #instructions is dependent on the compiler → not on the architecture

Let us look at IPC and frequency

IPC

- What is the IPC of an in-order pipeline?

1 if there are no stalls, otherwise < 1

Methods to increase IPC

- Forwarding
- Having more not-taken branches in the code
- Faster instruction and data memories

What about frequency?

What is frequency dependent on …

Frequency = 1 / clock period

Clock Period:

- 1 pipeline stage is expected to take 1 clock cycle
- Clock period = maximum latency of the pipeline stages

How to reduce the clock period?

- Make each stage of the pipeline smaller by increasing the number of pipeline stages
- Use faster transistors

Limits to Increasing Frequency

Assume that we have the fastest possible transistors

Can we increase the frequency to 100 GHz?



Reasons

Limits to increasing frequency - II

What does it mean to have a very high frequency?

Before answering, keep these facts in mind:

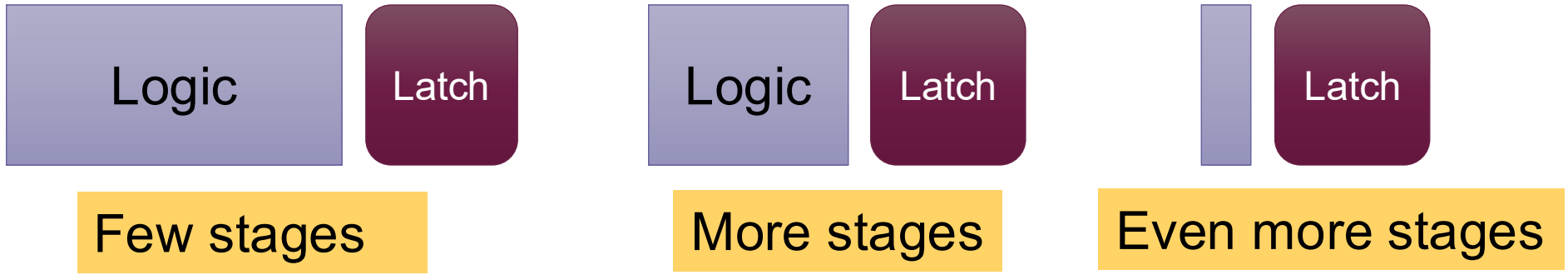
1. Thumb Rule: $P \propto f^3$

2. Thermo-dynamics: $\Delta T \propto P$

P → power
f → frequency
T → Temperature

3. We need to increase the number of pipeline stages → more hazards, more forwarding paths

How many pipeline stages can we have?



- We are limited by the latch delay
- Even with an infinite number of stages, the minimum clock period will be equal to the latch delay

Pipeline Stages vs IPC

$$CPI = \frac{1}{IPC}$$

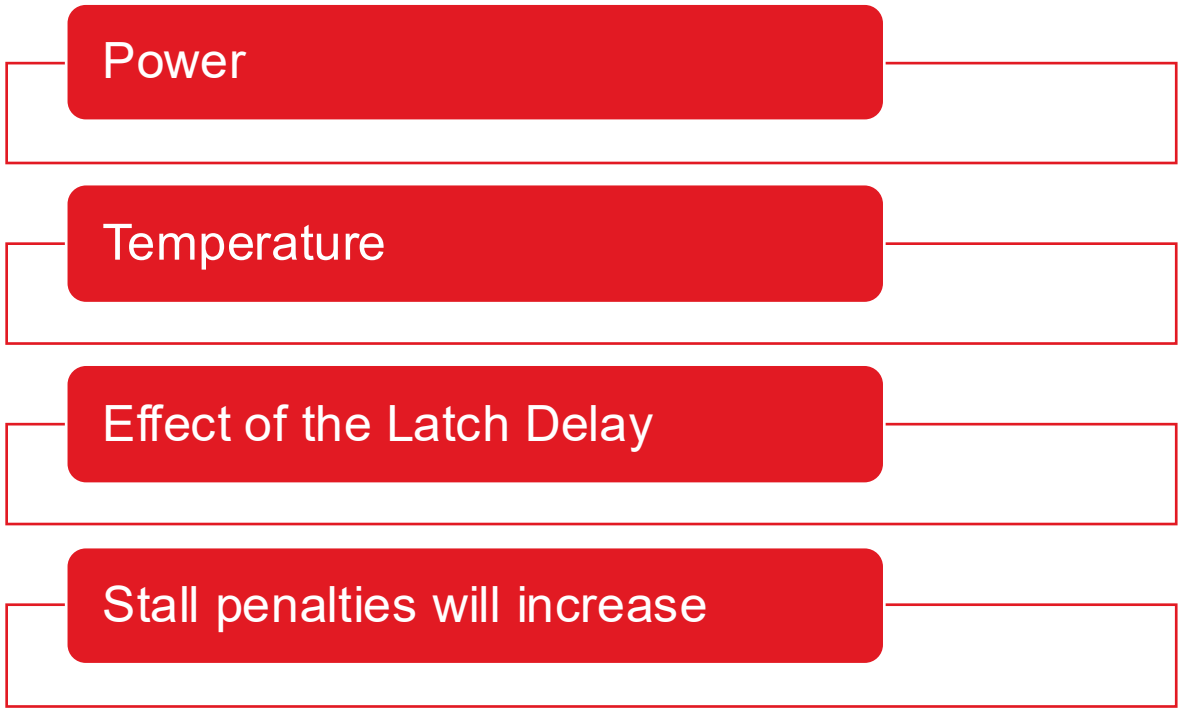
$$CPI = CPI_{ideal} + stall_rate * stall_penalty$$

- The stall rate will remain more or less constant per instruction with the number of pipeline stages
- The **stall penalty** (in terms of cycles) will however **increase**
- This will lead to a net increase in CPI and **loss** in IPC



As we increase the number of stages, the IPC goes down.

Summary: Why we cannot increase frequency by increasing the number of pipeline stages?



Since we cannot increase frequency …



Increase IPC

Increase IPC

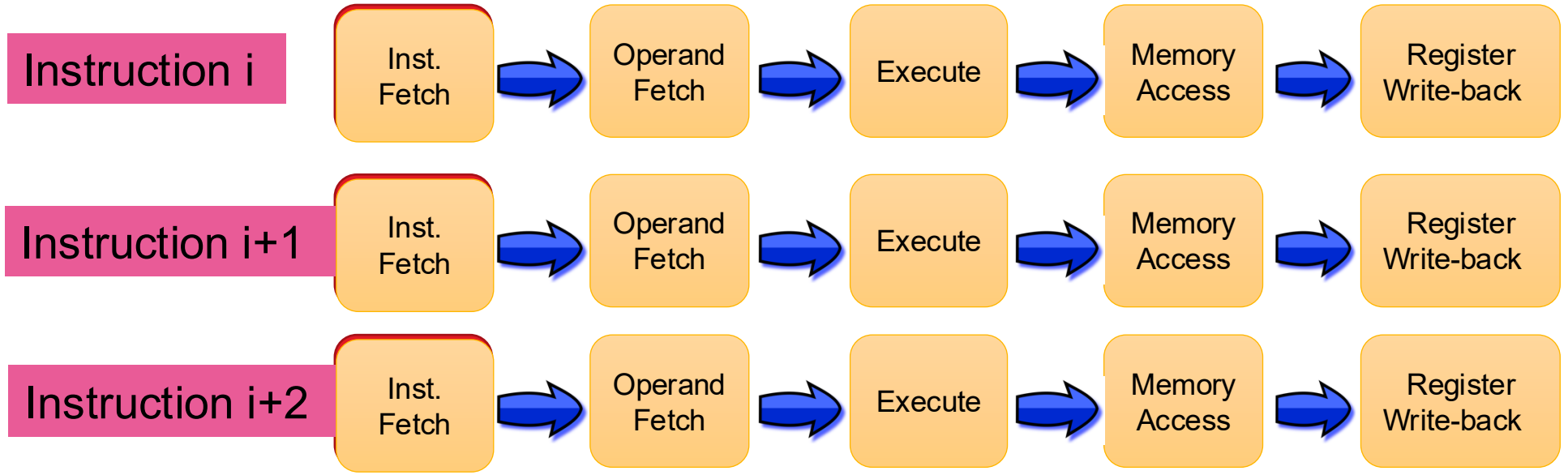
Issue more instructions per cycle

2, 4, or 8 instructions

Make it a **superscalar** processor → A processor that can execute multiple instructions per cycle

In-order Superscalar Processor

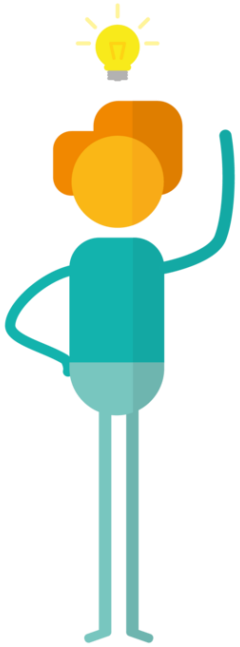
Have multiple in-order pipelines.



In-order Superscalar Processor - II

- There can be dependences between instructions
- Have $O(n^2)$ forwarding paths for an *n*-issue processor
- Complicated logic for detecting dependences, hazards, and forwarding
- Still might not be enough ...
- To get the peak IPC (= n) in an *n*-issue pipeline, we need to ensure that there are no stalls
- There will be no stalls if there are no taken branches, and no data dependences between instructions.
- Programs typically do not have such long sequences of instructions without dependences
- Load/store instn are difficult to handle

What to do ...



Don’t follow program order

```
mov r1, 1
add r3, r1, r2
add r4, r3, r2
mov r5, 1
add r6, r5, 1
add r8, r7, r6
```

Too many dependences

Execute out of order

Execute on a 2-issue OOO processor

```
mov r1, 1          mov r5, 1
add r3, r1, r2     add r6, r5, 1
add r4, r3, r2     add r8, r7, r6
```



Execute 2 instructions in parallel

Continuation ...

	issue slot 1	issue slot 2
cycle 1	mov r1, 1	mov r5, 1
cycle 2	add r3, r1, r2	add r6, r5, 1
cycle 3	add r4, r3, r2	add r8, r7, r6

In Out-of-order (OOO) processors, the execution is not as per program order. It is as per the data dependence order → the consumer is executed always after the producer.

Basic Principle of OOO Processors

Create a pool of instructions

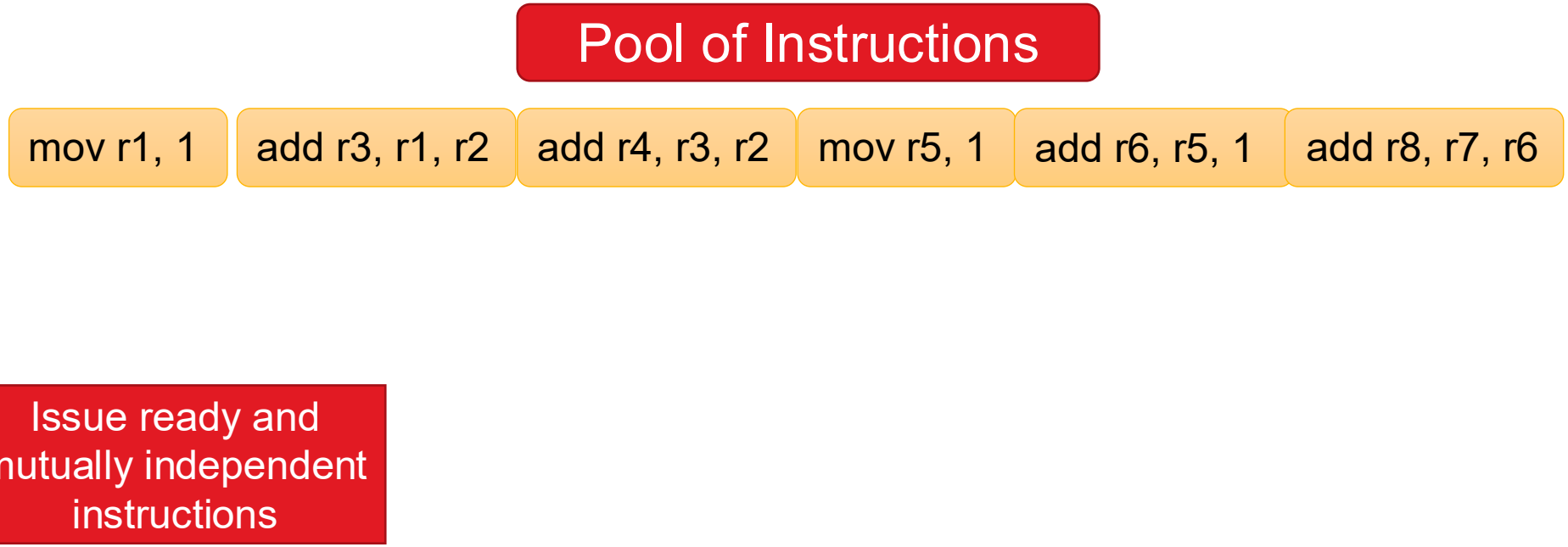
Find instructions that are mutually independent and have all their operands ready

Execute them out-of-order

ILP

Instruction level parallelism
The number of ready and independent instructions we can simultaneously execute.

Revisit the Example



Pool of Instructions: Instruction Window

- Needs to be large enough such that the requisite number of mutually independent instructions can be found.
- Typical instruction window sizes: 64 to 128
- How do we create a large pool of instructions in a program with branches? We need to be sure that **all the instructions** are on the correct path



```
for (i = 1; i < m; i++) {
        for (j = 1; j < i; j ++ ) {
                if (j %2 == 0) continue;
                ....
        }
}
```

Example

Problems with creating an Instruction Pool

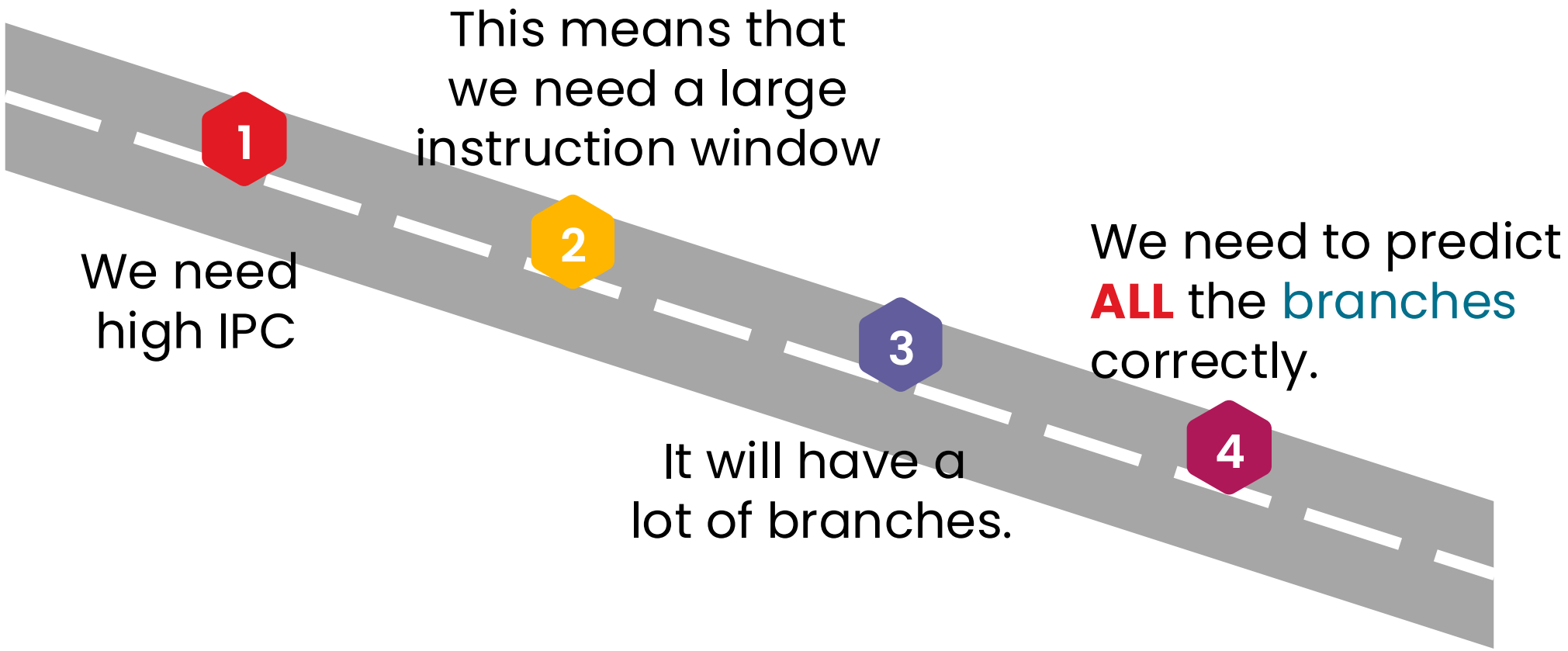


Typically 1 in 5 instructions is a branch



Predict the directions of the branches, and their targets

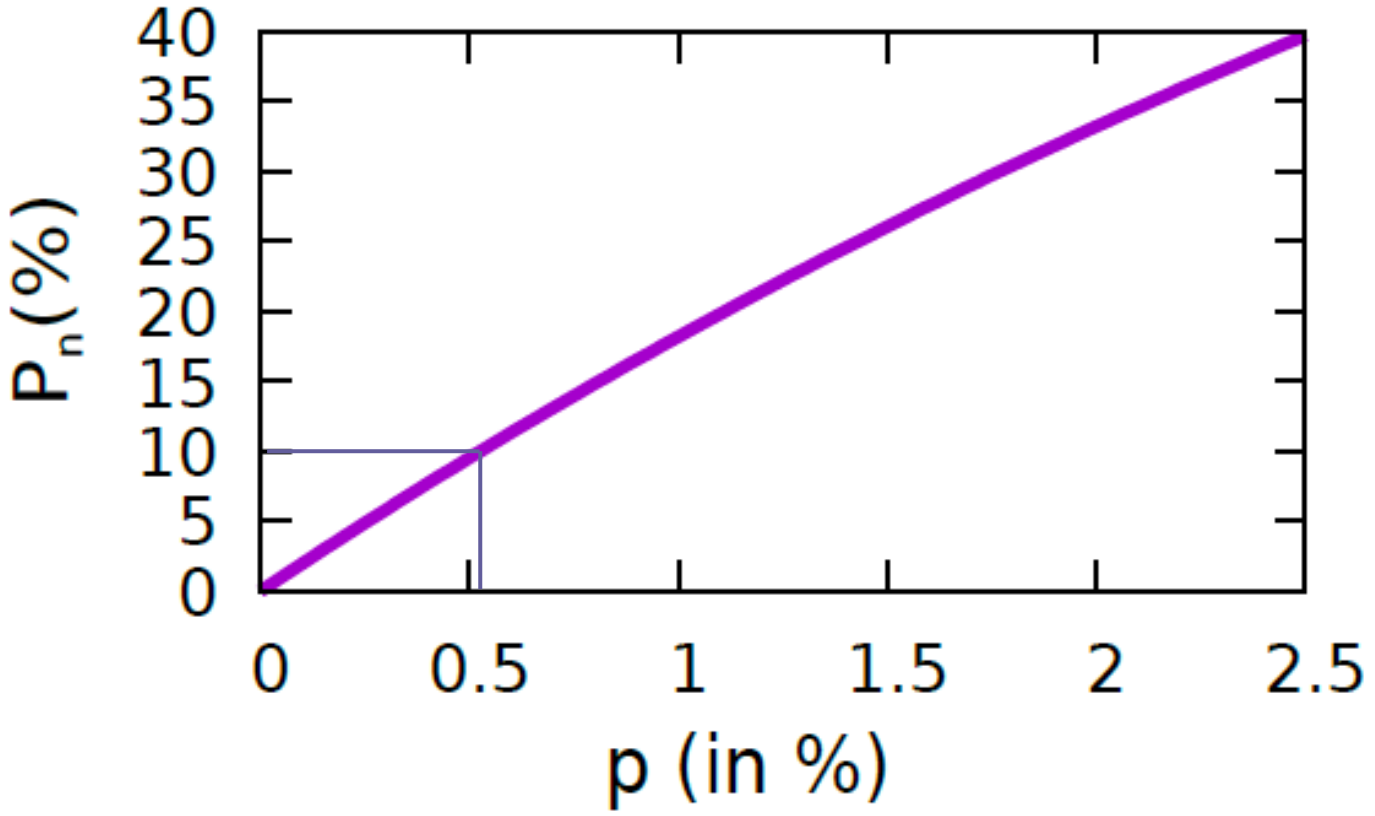
Motivation for Branch Prediction



The Maths of Branch Prediction

Number of instructions	n
Number of branches	**n/5**
Probability of predicting any given branch incorrectly	p
Probability of predicting **ALL the branches** correctly	$(1-p)^{\frac{n}{5}}$
Probability of making at least a single mistake (branch misprediction) in a pool of *n* instructions.	$P_n = 1-(1-p)^{\frac{n}{5}}$

For (n=100) : A plot of P_n vs p





If P_n = 10%, p has to be as low as 0.5% !!!



If we need a large instruction window, we need a very accurate branch predictor. The accuracy of the branch predictor limits the size of the instruction window.

Nature of Dependences

Dependences between Instructions

Program Order Dependence

```
mov r1, 1
mov r2, 2
```

- One instruction appears after the other in program order

The program order is the order of instructions that is perceived by a single cycle in-order processor executing the program.

Data Dependences

RAW → Read after Write Dependence (True dependence)

```
mov r1, 1
add r3, r1, r2
```

- It is a producer-consumer dependence.
- The earlier instruction produces a value, and the later instruction reads it.

Data Dependences - II

WAW → Write after Write Dependence (Output dependence)

```
mov r1, 1
add r1, r4, r2
```

- Two instructions write to the same location
- The later instruction needs to take effect after the former

Data Dependences - III

WAR → Write after Read Dependence (Anti dependence)

```
add r1, r2, r3
add r2, r5, r6
```

- Earlier instruction reads, later instruction writes
- The later instruction needs to execute after the earlier instruction has read its values

Control Dependences

```
beq .label
.....
.label
        add r1, r2, r3
```

- The *add* instruction is control dependent on the branch(*beq*) instruction
- If the branch is taken then only the *add* instruction will execute, not otherwise

Basic Results

In-order processors respect all program order dependences. Thus, they automatically respect all data and control dependences.

OOO processors respect only data and control dependences.

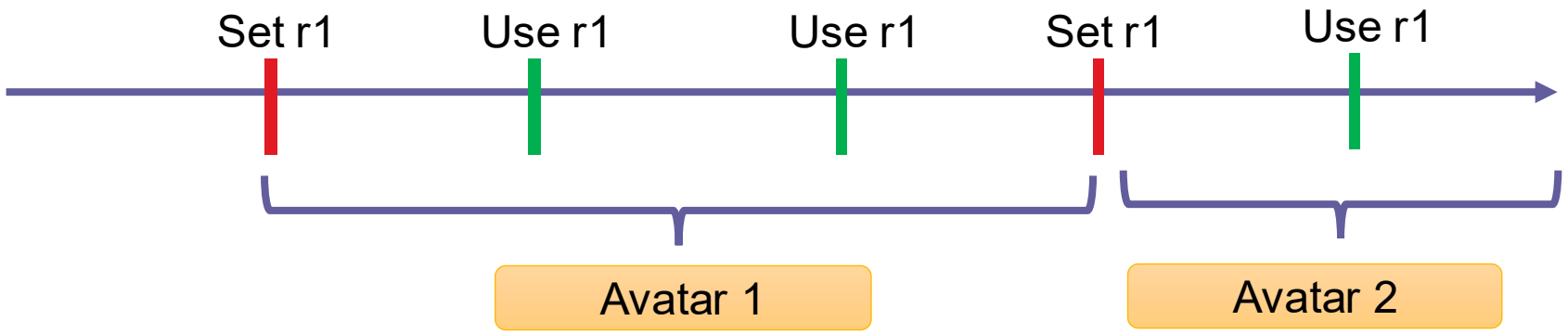
Can output and anti dependences be removed?

```
mov r1, 1
add r5, r6, r7
add r1, r4, r2
add r8, r9, r10
```

```
add r1, r2, r3
add r5, r6, r7
add r2, r5, r6
add r8, r9, r10
```



- Don’t you think that these dependences are there because we have a finite number of registers.
- What if we had an infinite number of registers?



Solution: Assume infinite number of physical registers

Architectural register → Physical register

Format in this example: rx is mapped to px<avatar number>

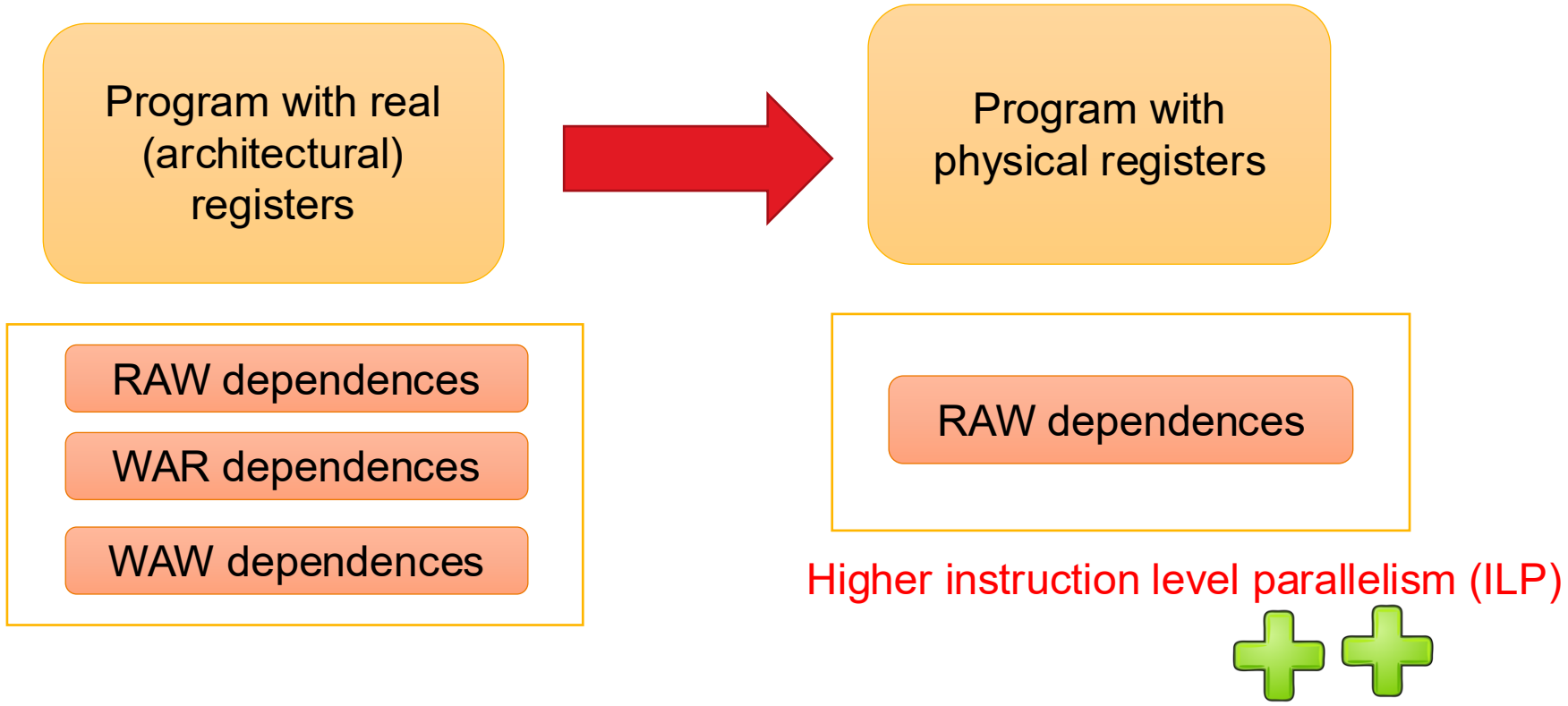
```
mov r1, 1
add  r1, r2, r3
add  r4, r1, 1
mov r2, 5
add r6, r2, r8
mov r1, 8
add r9, r1, r2
```

Code with architectural registers

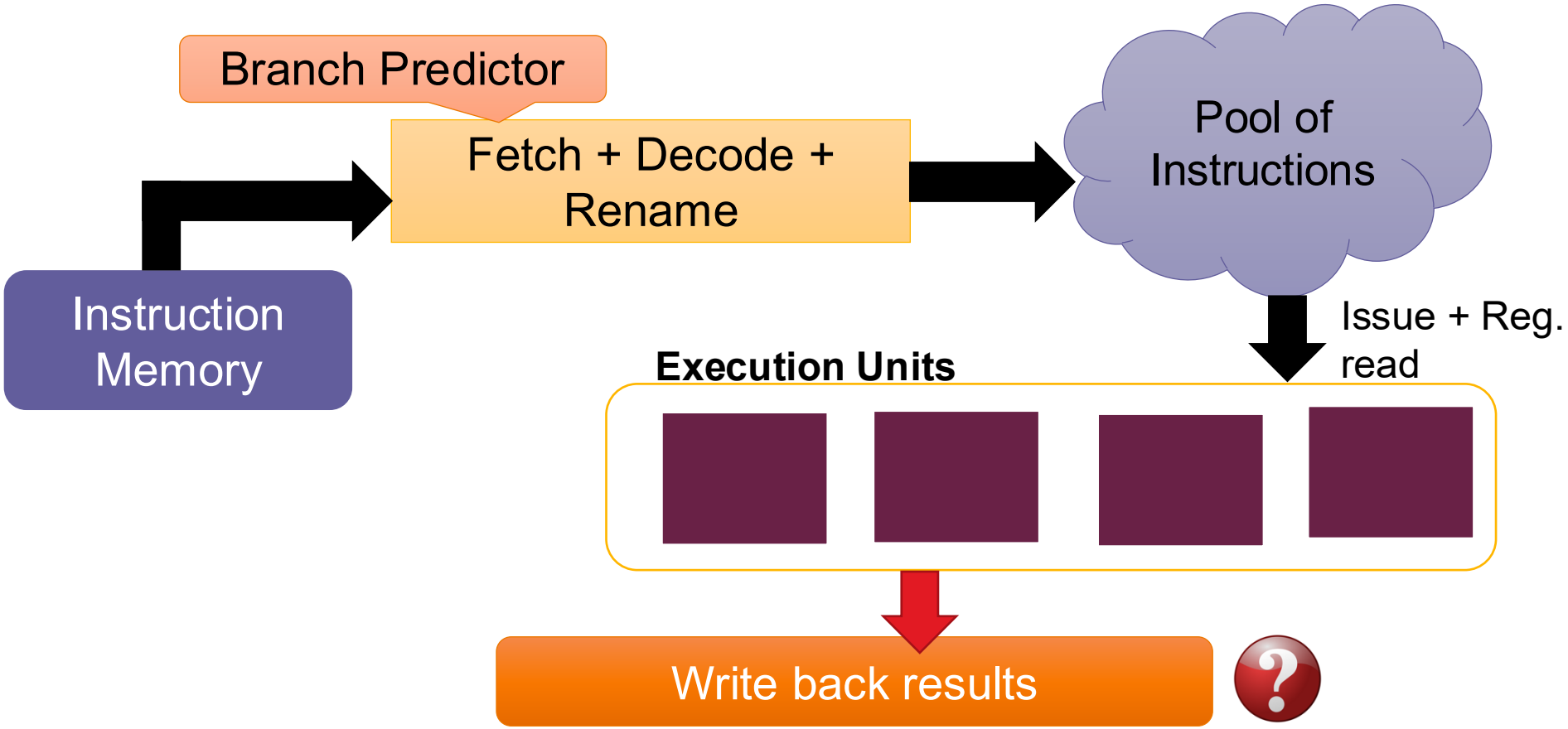
```
mov p11, 1
add  p12, p2, p3
add  p41, p12, 1
mov p21, 5
add p61, p21, p8
mov p13, 8
add p91, p13, p21
```

Code with physical registers

Renaming


Program with real (architectural) registers
Program with physical registers
RAW dependences
WAR dependences
WAW dependences
RAW dependences
Higher instruction level parallelism (ILP)

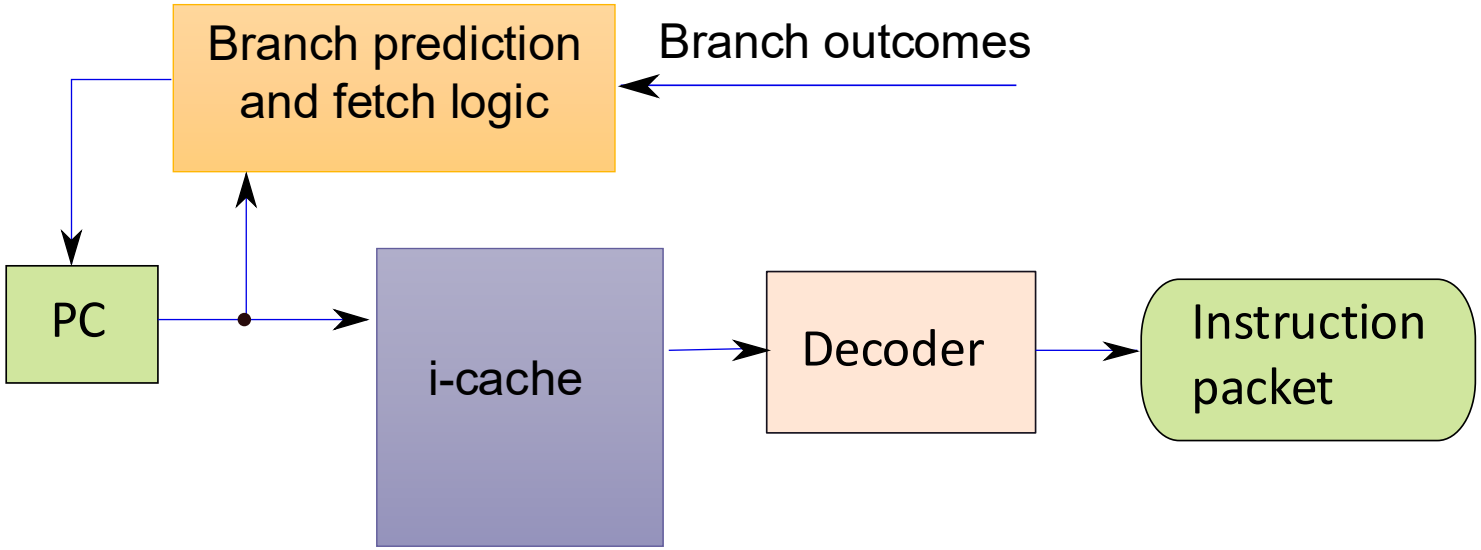
Where are we now ...



The Fetch and Decode Stages

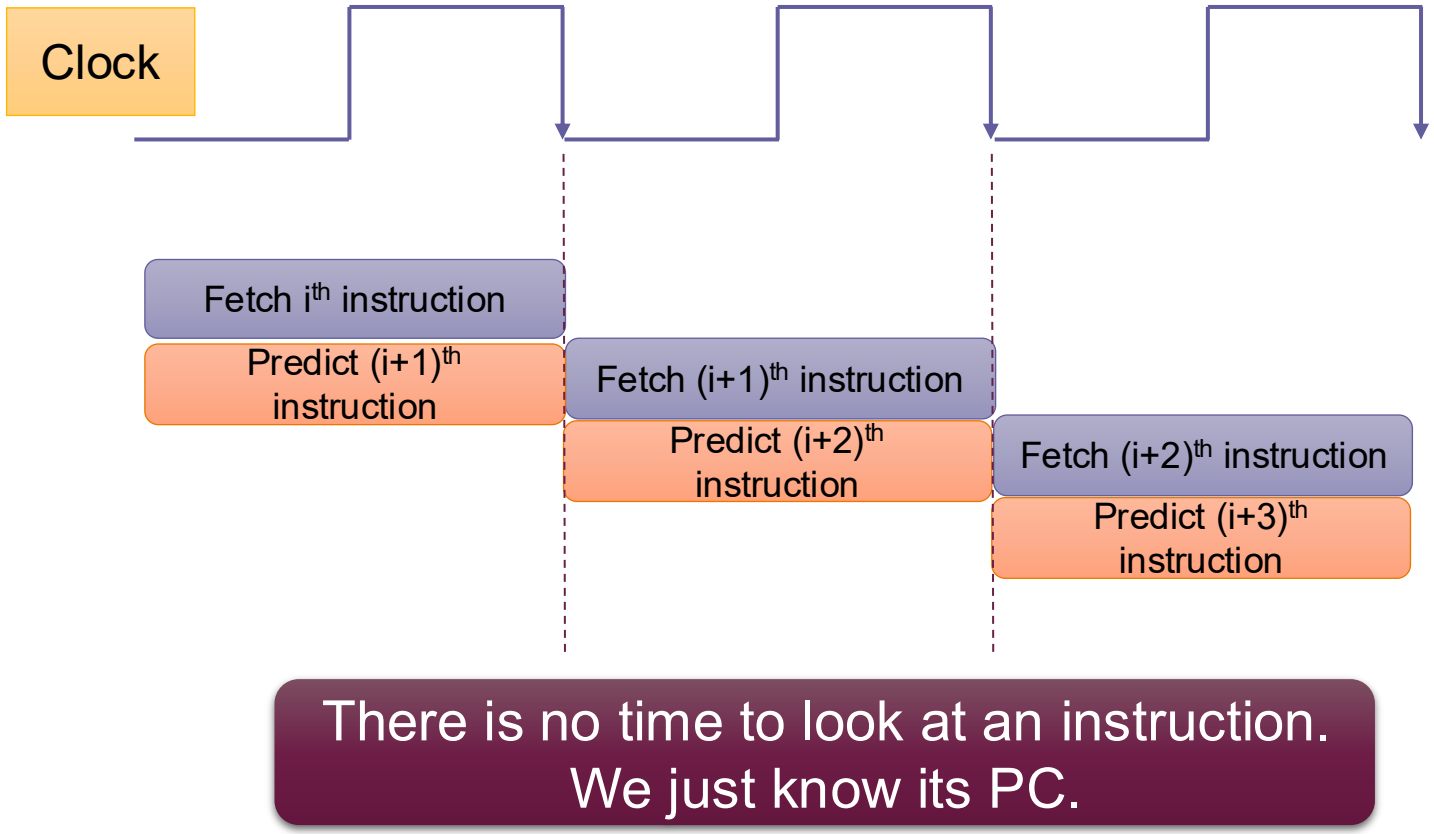
Background

The Fetch Stage



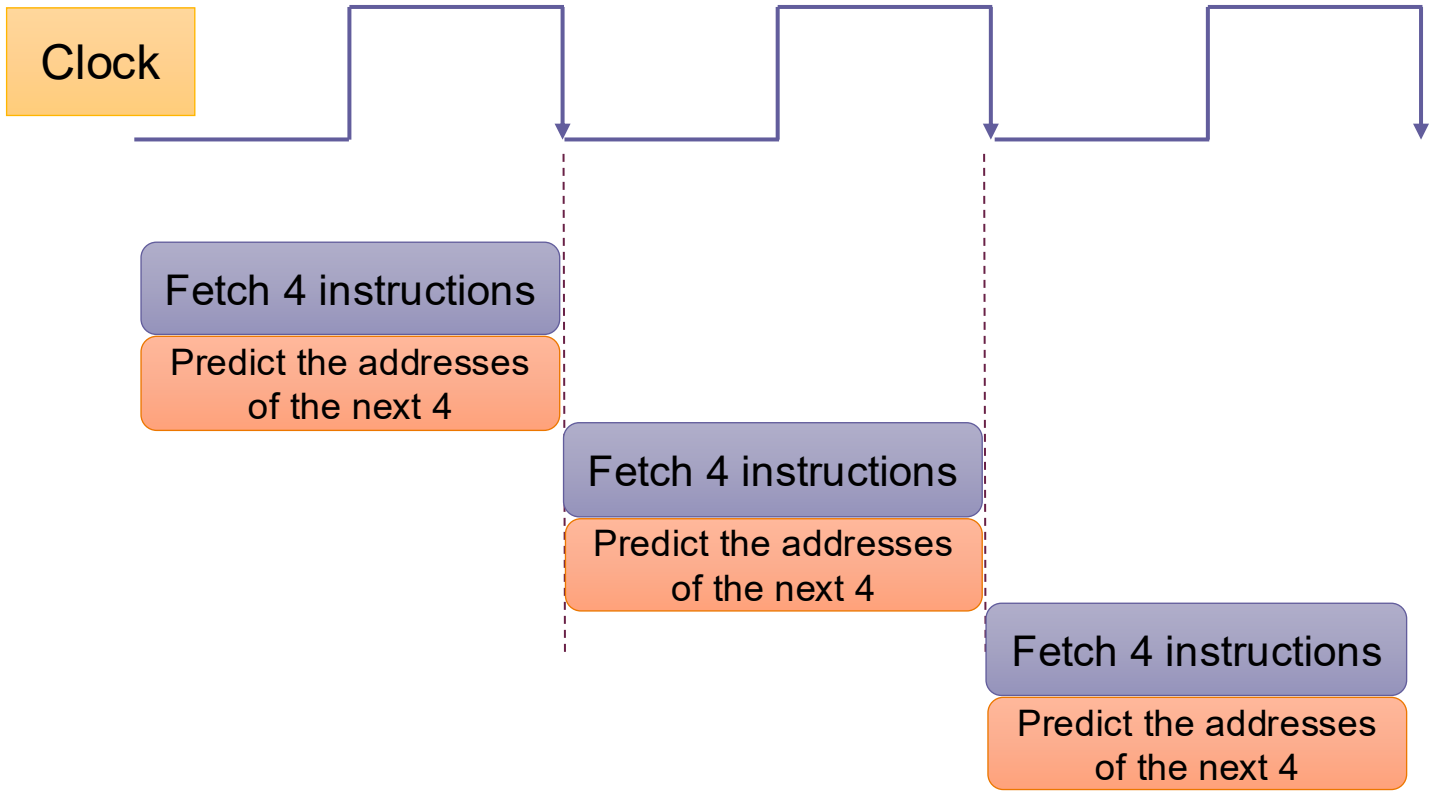
- The branch prediction and fetch logic computes the addresses of the instructions to be fetched in the next cycle
- The corresponding instructions are fetched from the i-cache
- They are sent down the pipeline

Timing: Fetching a single instruction per cycle



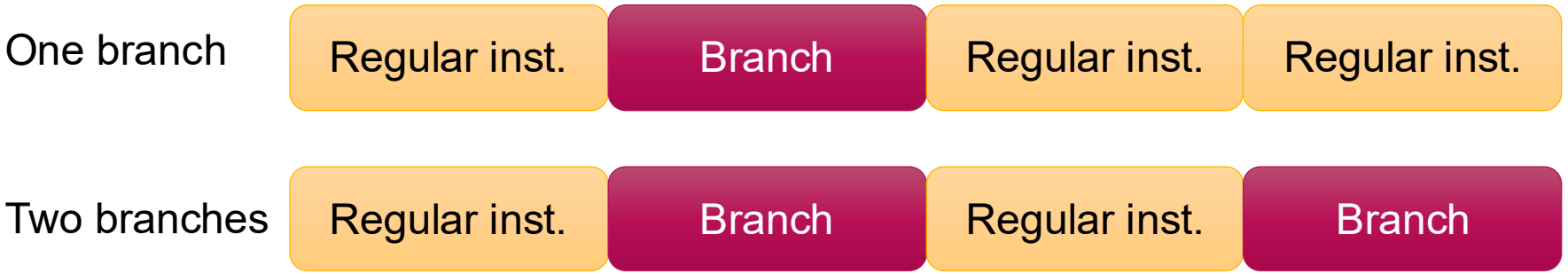
There is no time to look at an instruction. We just know its PC.

Timing: Fetching 4 instructions simultaneously



This is very hard to do. Why?

Branches in a Bundle of 4 Instructions



- If we have branches in a bundle of 4 instructions, the instructions will not be contiguous
- To predict the second branch, we need to first predict the first branch

Branch Prediction: Three Problems

- Predict if an instruction is a branch or not.
- If it is a branch, predict is direction.
- Predict its target.



Contents



1. Prediction of the Instruction Type
2. Prediction of the Branch Outcome
3. Prediction of the Branch Target
4. Decode Stage

Is an instruction a branch or not?

Insights

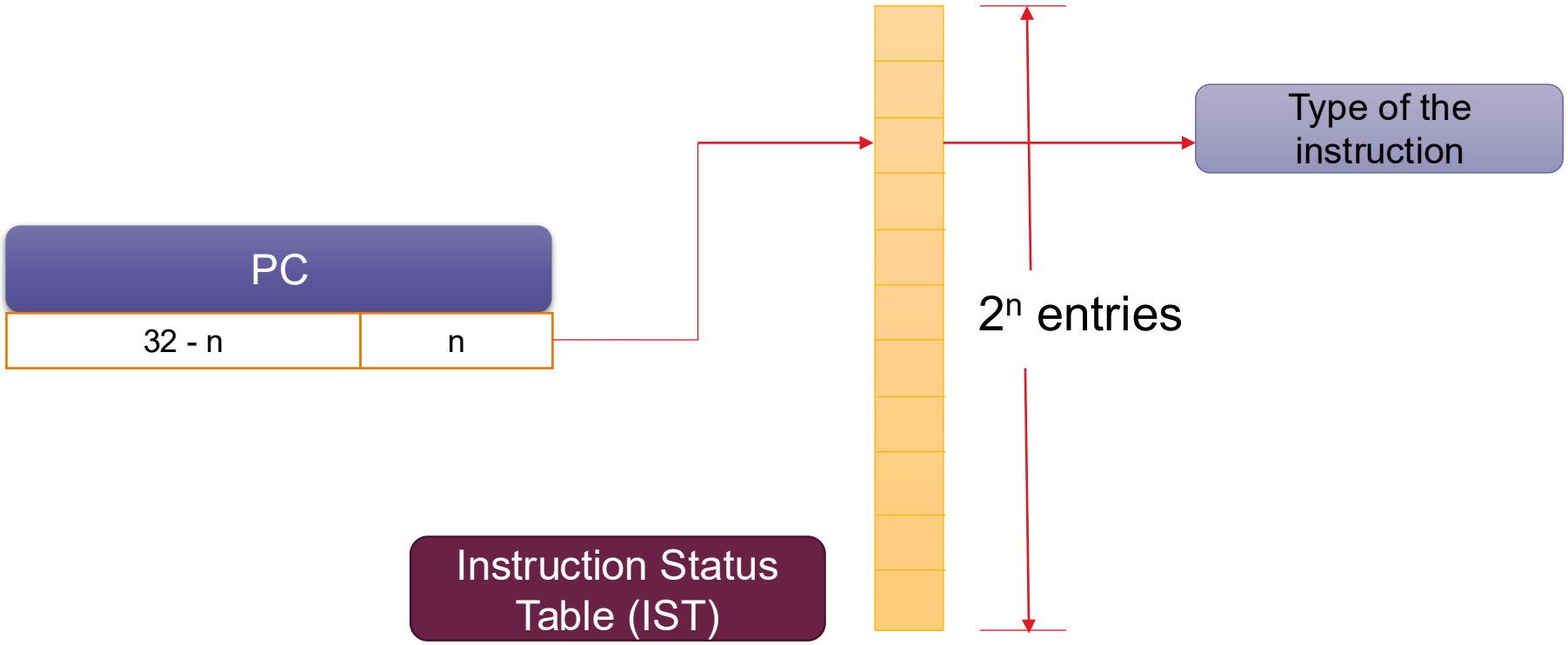
- Given a PC, the status of the instruction (branch or not) does not change.
- Can we use this information?



Approach

- The last time that we had seen a branch remember its PC
- Next time we see a PC, **check** if we have seen it before
- Also remember the type of the branch:
  - Unconditional branch
  - Conditional branch → depends on the result of a previous compare instruction
  - Function call
  - Return

Make a structure in hardware to remember ...



Assumption	Assume a 32-bit PC address

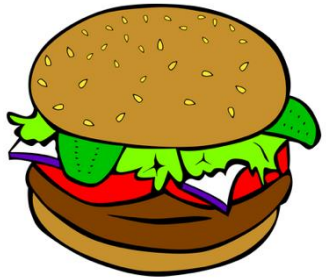
Basic Method of Operation

When we see a branch instruction

- Access its corresponding entry in the table
- Record the branch type

When we see an instruction:

- Check the corresponding entry of the table
- Read the type of the instruction



Why do we choose the *n* least significant bits?

Instruction Status Table (IST)

If we choose the least significant 10 bits of the PC address, we will have 1024 entries in the IST

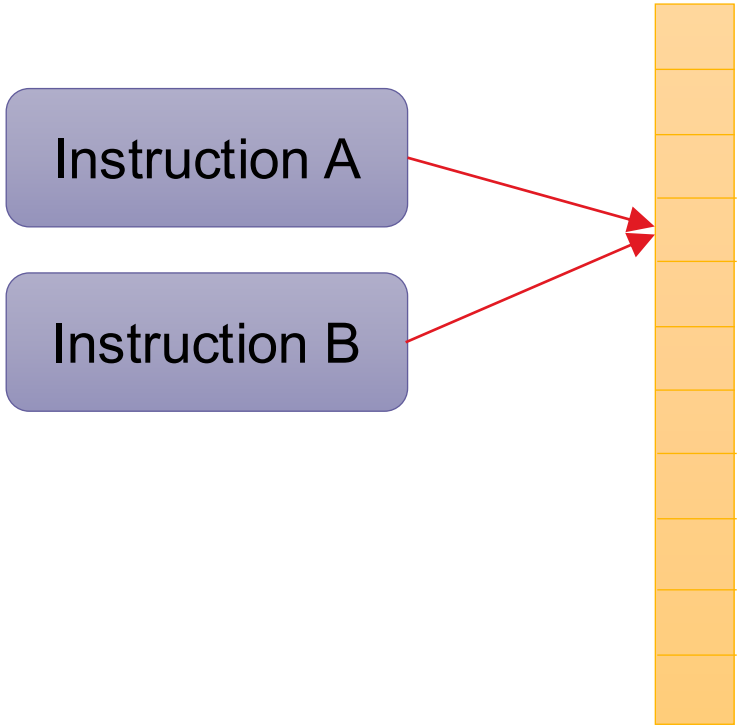
Why 10 bits?

For a 32-bit PC, we cannot have a 2^{32} entry table.

- Too big
- Too slow
- Too much of power
- Too much of area

We need to manage with a smaller table

Destructive Interference



Two instructions can map to the same entry; same last *n* bits.

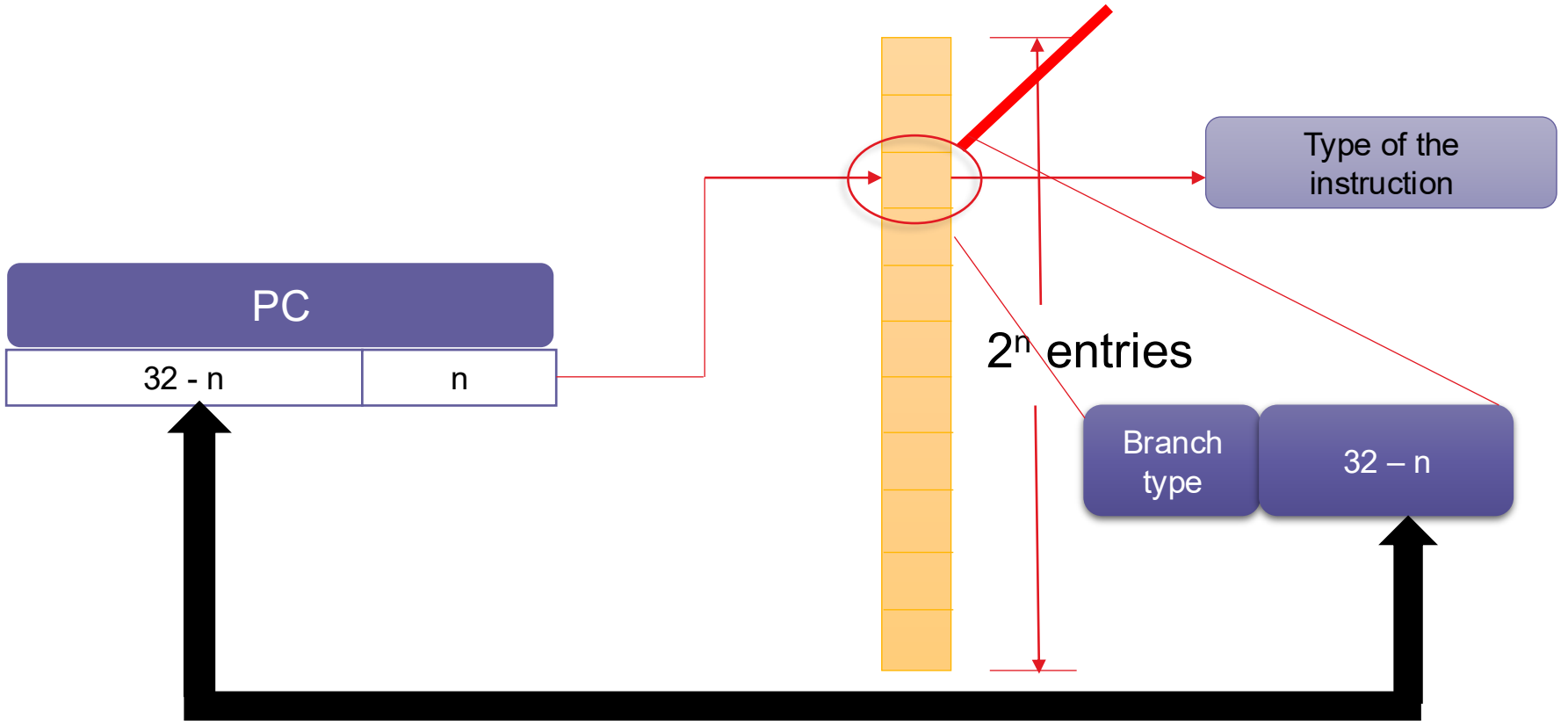
Defined as **destructive inteference**

Branch Aliasing

- Possible for a branch and a non-branch instruction to map to the same entry
- Possible for two branches to map to the same entry
- Possible for two non-branches to map to the same entry
- Need for disambiguation
  - Augment each entry of the IST

Disambiguation Between Addresses

Add a (*32-n)* bit tag



Disambiguation

- We can keep the status of only branches in the IST
- However, for every instruction, we need to check the IST
- If there is no entry, then we need to predict that it is not a branch
- Can be wrong
- What to do ☹

Why does an IST work?

- Mainly because most pieces of code exhibit temporal locality
- In a given window of time instructions tend to repeat themselves

Contents



1. Prediction of the Instruction Type
2. Prediction of the Branch Outcome
3. Prediction of the Branch Target
4. Decode Stage

Predict the Direction of the Branch (Outcome)



- If a branch is unconditional → no need to predict (taken)
- If a branch is a call/return → no need to predict (taken)
- If a branch is conditional → need to predict

Example Code

C

```
void foo() {
        for (i=0; i < 5; i++) {
                ...
        }
}
```

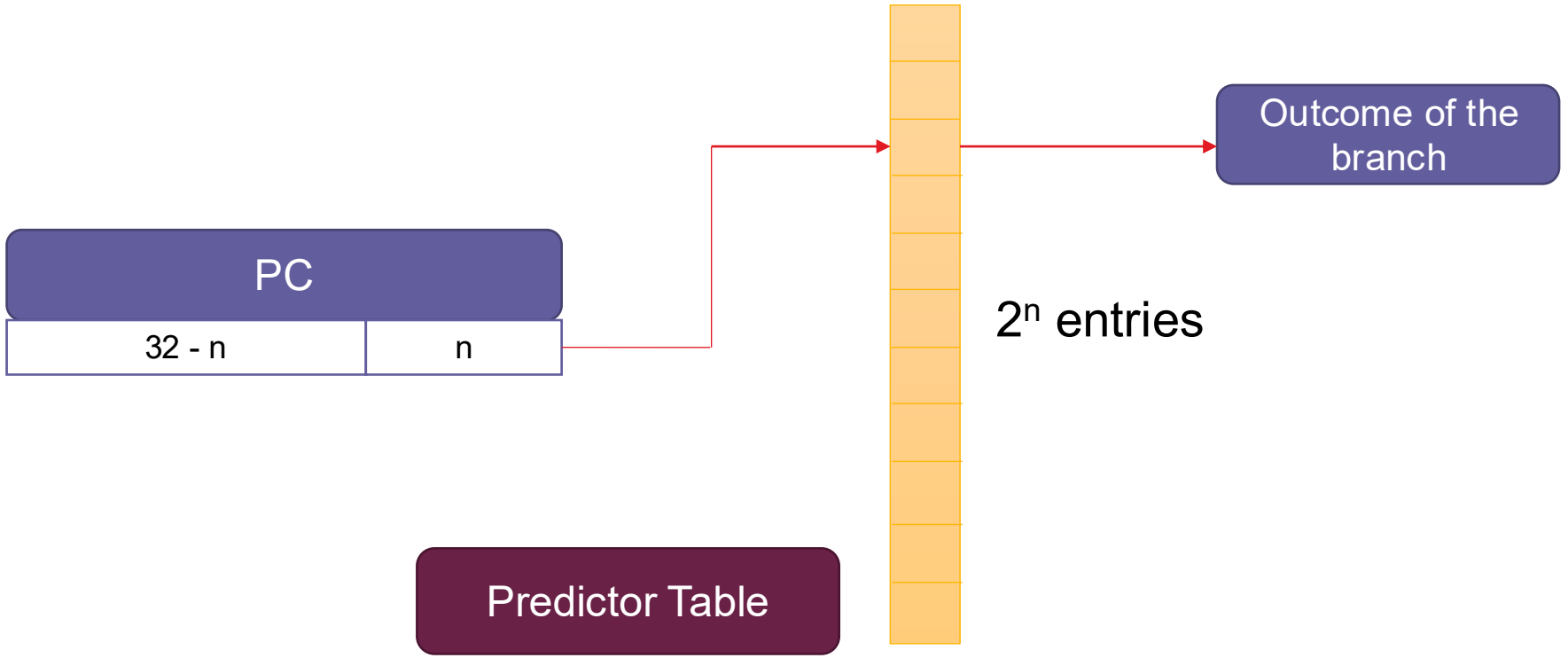
Assembly

```
.foo:
        mov   r0, 0
.loop:  cmp   r0, 5
        beq   .exit
        add   r0, r0, 1
        b .loop
```

Predict

This branch has roughly similar behavior most of the time. Predominantly not taken.

Simple Bimodal Predictor



Remember the last outcome.
Current prediction = last outcome

Bimodal Predictor - II

Each entry saves the last recorded outcome of the branch

- taken or not-taken

What is the problem:

- The *beq* instruction is evaluated 6 times
- Assume the default is not-taken
- First 5 times → correctly predicted not-taken
- 6th and last time → incorrectly predicted to be taken

Now assume we call the function foo() once again

- The first time → we will predict taken. This is wrong
- Again the last time will be wrong
- Misprediction rate: 2/6

Increasing Accuracy

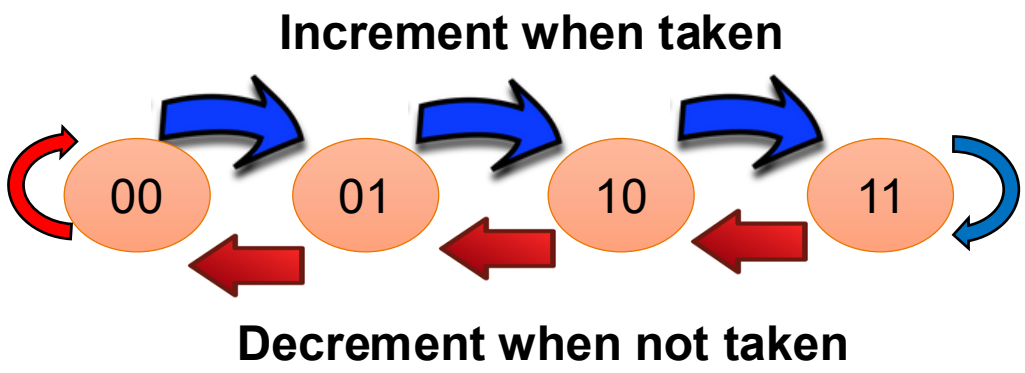
What is the problem?

- The *beq* instruction is fundamentally biased towards not-taken
- One exception at the end of the *for* loop cannot change its inherent behavior

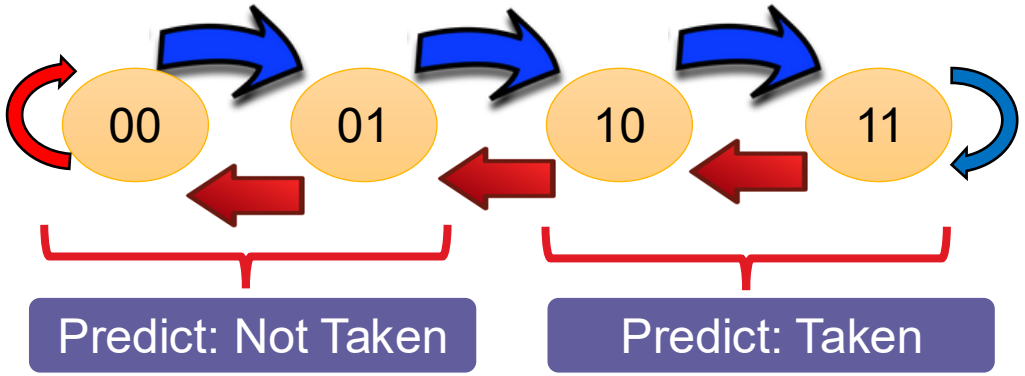
Let us add some hysteresis

- Instead of having a single bit in each entry, have a 2-bit counter

Saturating Counter



Saturating Counters



Algorithm for updates:

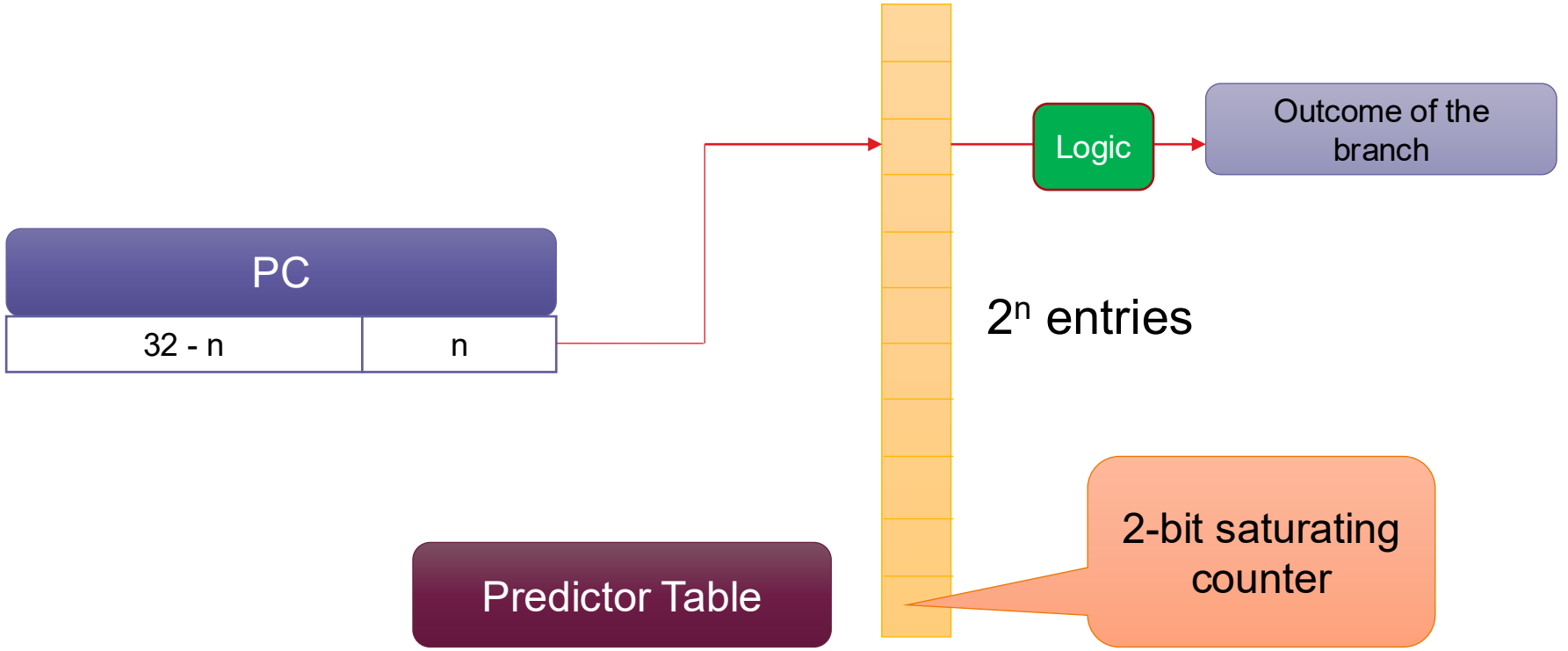
- If a branch is taken, increment the saturating counter
- If it is not taken, decrement the counter

For prediction:

- 00 and 01 → not taken
- 10 and 11 → taken

State	Name
00	Strongly not taken
01	Weakly not taken
10	Weakly taken
11	Strongly taken

Bimodal Predictor with Saturating Counters



Will it help?

```
.foo:
        mov   r0, 0
.loop:  cmp   r0, 5
        beq   .exit
        add    r0, r0, 1
        b .loop
```

1st Time: Predict not-taken. Correct. Starts with 01. Moves to 00

2nd Time: Predict not-taken. Correct. Remains at 00

...

6th Time: Predict not-taken. Wrong. Starts with 00. Moves to 01



Misprediction rate: Down from 2/6 to 1/6

Why do saturating counters work?

- In this case, we have a degree of hysteresis. The status of the branch (*beq .exit*) moves between the strongly not taken and weakly not taken states.
- If a branch is strongly biased towards one direction, but once in a while changes its direction, we should use saturating counters.
- They capture stable behavior with occasional anomalies.